

Formalizing Quantifier Elimination

A Lean formalization in model theory

Model Theory

Language

- A first-order language specifies relation and function symbols.
- Each symbol has a fixed arity.
- DLO language: one binary relation symbol, interpreted as $\leq$.

Structure

- A structure interprets the symbols of a language on a domain.
- For the order language, a structure is a type M with a binary relation.
- Example: $(\mathbb{Q}, \leq)$.

Terms

- Terms are built from variables and function symbols.
- In the pure order language, there are no function symbols.
- DLO terms are just variables or parameters: x, y, a, b .

Formula

- Formulas are built from atomic formulas using logical connectives and quantifiers.
- Atomic DLO examples: $x \leq y$, $x = y$.
- Formula example: $\exists z (x < z \wedge z < y)$.

Sentence

- A sentence is a formula with no free variables.
- Example: $\forall x \forall y (x \leq y \vee y \leq x)$.
- Density: $\forall x \forall y (x < y \rightarrow \exists z (x < z \wedge z < y))$.

Satisfaction

- $M \models \varphi[a]$: φ holds in M under assignment a .
- For sentences, no assignment is needed.
- Example: $(\mathbb{Q}, \leq) \models \exists z (0 < z \wedge z < 1)$.

Theory

- A theory is a set of sentences.
- The theory DLO contains the axioms for:
 - linear orders,
 - density,
 - no endpoints.

Model

- $M \models T$ means M satisfies every sentence in T .
- $(\mathbb{Q}, \leq)$ is a model of DLO.
- $(\mathbb{Z}, \leq)$ is not: it is not dense.

Type Theory

Judgements

- $a : A$ is a judgement: a has type A .
- Every expression has a type.
- Definitional equality is written $a \equiv b$.

Propositions as Types

- A proposition is viewed as a type of evidence.
- A proof of P is a term $p : P$.
- To prove a theorem is to construct an object of the corresponding type.

Constructive Reasoning

- A proof of $\exists x, P(x)$ gives a witness and evidence.
- A proof of $P \vee Q$ chooses one side and proves it.
- The law of excluded middle, $P \vee \neg P$, is not built in constructively.
- Lean allows classical reasoning when explicitly invoked.

Curry-Howard Isomorphism

$$\llbracket P \Rightarrow Q \rrbracket \equiv \llbracket P \rrbracket \rightarrow \llbracket Q \rrbracket$$

$$\llbracket P \wedge Q \rrbracket \equiv \llbracket P \rrbracket \times \llbracket Q \rrbracket$$

$$\llbracket \text{True} \rrbracket \equiv 1$$

$$\llbracket P \vee Q \rrbracket \equiv \llbracket P \rrbracket + \llbracket Q \rrbracket$$

$$\llbracket \text{False} \rrbracket \equiv 0$$

$$\llbracket \forall x : A. P \rrbracket \equiv \prod x : A. \llbracket P \rrbracket$$

$$\llbracket \exists x : A. P \rrbracket \equiv \sum x : A. \llbracket P \rrbracket$$

Lean

```

/-- A formula is equivalent to a quantifier-free formula over a theory. -/
def IsQFEquivalent (T : L.Theory) {α : Type*} (φ : L.Formula α) : Prop :=
  ∃ ψ : L.Formula α, ψ.IsQF ∧ φ ⇔[T] ψ

/-- A theory has quantifier elimination if every formula is equivalent,
over the theory, to a quantifier-free formula. -/
def HasQuantifierElimination (T : L.Theory) : Prop :=
  ∀ {α : Type} (φ : L.Formula α), T.IsQFEquivalent φ

```

```
-- The theory of dense linear orders without endpoints
has quantifier elimination. -/
theorem dlo_hasQuantifierElimination :
  Language.order.dlo.HasQuantifierElimination := by
  ...
```